



INSTITUTO POLITECNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Individual:

FUNCIONES DE ACTIVACIÓN

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Guevara Badillo Areli Alejandra

Grupo: 6CM3

lunes, 8 de diciembre de 2025

■ ¿QUÉ SON LAS FUNCIONES DE ACTIVACIÓN Y PARA QUÉ SIRVEN?

Para entender qué es una función de activación, primero imaginemos una neurona artificial. Esta recibe datos, los mezcla (suma ponderada) y obtiene un resultado numérico. Sin embargo, ese número crudo no siempre es útil por sí mismo. Aquí entra la función de activación.

¿Qué es?

Es una fórmula matemática que se coloca al final de la neurona. Es el "guardián" o el "interruptor" que decide qué información es importante y debe pasar a la siguiente capa de la red.

¿para qué sirve? Tiene dos propósitos vitales:

1. Decidir el encendido (Output):

Transforma la suma matemática interna en una salida comprensible. Por ejemplo, puede convertir un número gigante como \$450\$ en un simple \$1\$ (encendido) o en un \$0.99\$ (99% de probabilidad). Sin ella, los números crecerían sin control.

2. Introducir "No Linealidad" (La clave de la inteligencia):

- Sin funciones de activación, una red neuronal, por muy grande que sea, solo podría aprender líneas rectas (como una regresión lineal simple).
- El mundo real es complejo (curvo, irregular). Las funciones de activación permiten que la red "doble" y "retuerza" sus líneas para ajustarse a datos complicados, como reconocer una cara o entender una frase.

Para que la red aprenda (usando un método llamado retropropagación), estas funciones deben tener una "pendiente" (derivada) calculable. Esto le dice a la red hacia dónde ajustar sus errores. Si la función es plana, la red deja de aprender (un problema conocido como "desvanecimiento del gradiente").

FAMILIAS DE FUNCIONES

El texto clasifica las funciones según cómo han evolucionado para resolver problemas de aprendizaje.

A. Las Clásicas (Curvas tipo "S")

Incluyen la Sigmoides y la Tangente Hiperbólica (Tanh).

- **Cómo funcionan:** Aplastan cualquier número para que quede entre 0 y 1 (Sigmoides) o -1 y 1 (Tanh).
- **El problema:** Sufren de "saturación". Si el número es muy alto o muy bajo, la curva se vuelve plana. Cuando la curva es plana, la red deja de aprender (el gradiente se desvanece). Por eso ya casi no se usan en capas ocultas profundas.

B. La Revolución de la Rectificación (ReLU)

La función ReLU cambió todo.

- **Cómo funciona:** Es muy simple. Si el número es positivo, lo deja pasar tal cual. Si es negativo, lo convierte en cero.
- **La ventaja:** Al no tener una curva suave en la parte positiva, no se "aplana", permitiendo entrenar redes muy profundas sin que el aprendizaje se detenga. Es el estándar actual.
- **El problema:** A veces "mata" neuronas (si siempre da 0, la neurona nunca se reactiva). Variantes como Leaky ReLU o ELU intentan arreglar esto permitiendo pequeños valores negativos.

C. La Nueva Ola (Suaves y Modernas)

Funciones como Swish, GELU y Softplus.

- **Cómo funcionan:** Son una mezcla entre la potencia de ReLU y la suavidad de las curvas clásicas.
- **Uso:** Se usan en los modelos más avanzados de hoy, como los Transformers (la tecnología detrás de ChatGPT) y visión por computadora moderna, ya que manejan mejor la complejidad que la rígida ReLU.

TABLA COMPARATIVA DE FUNCIONES DE ACTIVACIÓN

Función de Activación	Fórmula	Características Principales	¿Dónde se usa comúnmente?
Lineal (Identidad)	$f(x) = x$	Deja pasar la señal sin cambios. No aprende patrones complejos.	Capa de salida en problemas de regresión (predecir precios, temperatura).
Sigmoide (Logística)	$\sigma(x) = \frac{1}{1 + e^{-x}}$	Convierte todo a un rango de 0 a 1. Se "aplana" en los extremos (problema de aprendizaje).	Capa de salida para clasificación binaria.
Tanh (Tangente Hiperbólica)	$\tanh(x)$	Similar a la Sigmoide, pero va de -1 a 1. Está centrada en cero, lo que ayuda al cálculo.	Capas ocultas en redes recurrentes (RNN) o redes pequeñas.
ReLU	$\max(0, x)$	Si es positivo pasa, si es negativo es 0. Muy rápida y eficiente.	Estándar universal para capas ocultas en casi todas las redes modernas (CNNs, MLPs).
Leaky ReLU	$x \text{ (si } x \geq 0)$ $\alpha x \text{ (si } x < 0)$	variante de ReLU. Deja pasar una pequeña señal negativa para evitar que las neuronas "mueran".	Redes profundas inestables o GANs (redes generativas).
ELU (Exponencial)	$x \text{ (si } x > 0)$ $\alpha(e^x - 1) \text{ (si } x \leq 0)$	Usa una curva suave para los negativos. Aprende rápido pero es más lenta de calcular.	Redes que necesitan aprender muy rápido y robustamente.
Softplus	$\ln(1 + e^x)$	Una versión suave y curva de ReLU. Nunca es exactamente cero.	Modelos probabilísticos o estadísticos específicos.
Swish / SiLU	$x \cdot \sigma(x)$	Curva moderna y sofisticada. Supera a ReLU en redes muy complejas.	visión por computadora avanzada (YOLO, EfficientNet) y Transformers.
GELU	$x \cdot \Phi(x)$	Función probabilística suave. Es la clave de los modelos de lenguaje actuales.	Transformers y LLMs (BERT, GPT-3, GPT-4).
Softmax	$\frac{e^{x_i}}{\sum e^{x_j}}$	Toma una lista de números y los convierte en probabilidades que suman 1.	Capa de salida obligatoria para clasificación de múltiples clases.

FUNCIONES DE ACTIVACIÓN CLÁSICAS

En la entrada, una neurona artificial calcula una 'suma ponderada' de su entrada y agrega un sesgo. Como antes, sea $z = \sum_i x_i u_i + b, z \in R$. Una vez calculado el valor z , la neurona artificial decide si se debe "disparar" o no, basándose en el valor de z .

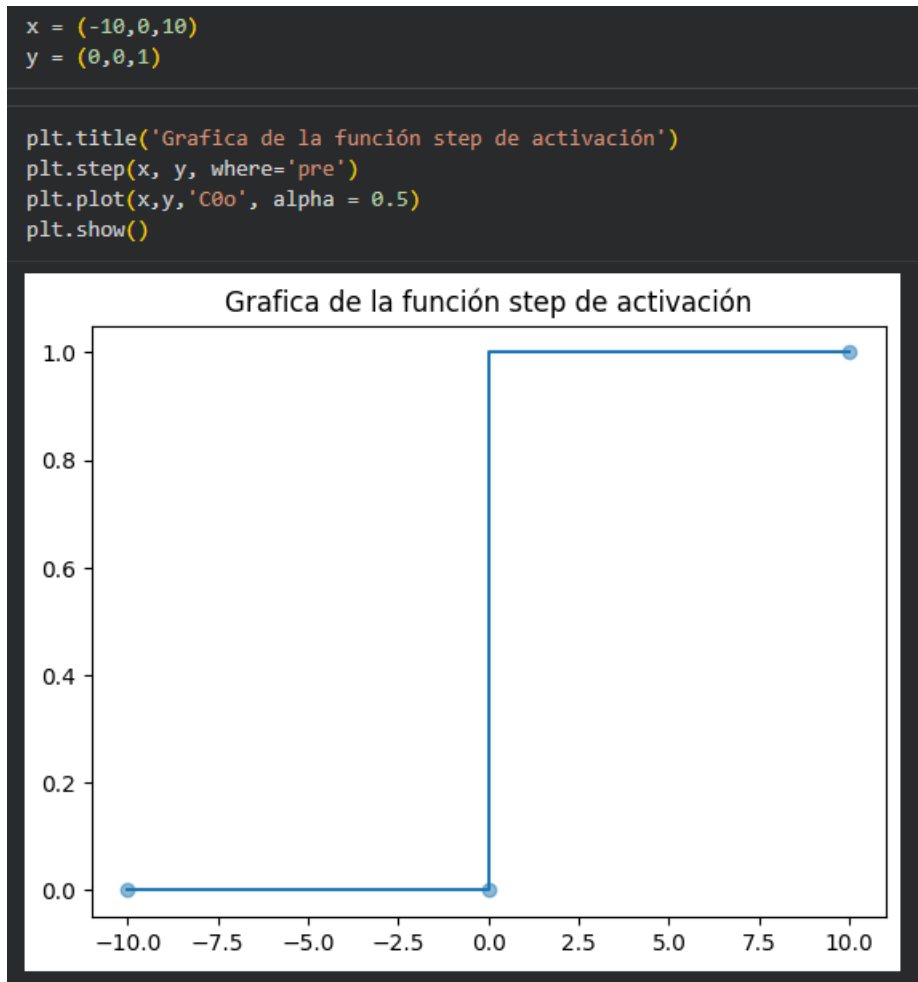
Función Step

Una función de activación basada en umbrales.

Si el valor de z está por encima de cierto valor, declare que se ha activado. Si es menor que el umbral, diga que no. El umbral más común es 0. La función de activación Step se define como:

$$f(z) = \text{Step}(z) \begin{cases} 1, & \text{if } x \geq 0, \\ 0, & \text{en otro caso} \end{cases}$$

Obviamente $f'(x) = 0$ for $x \neq 0$.



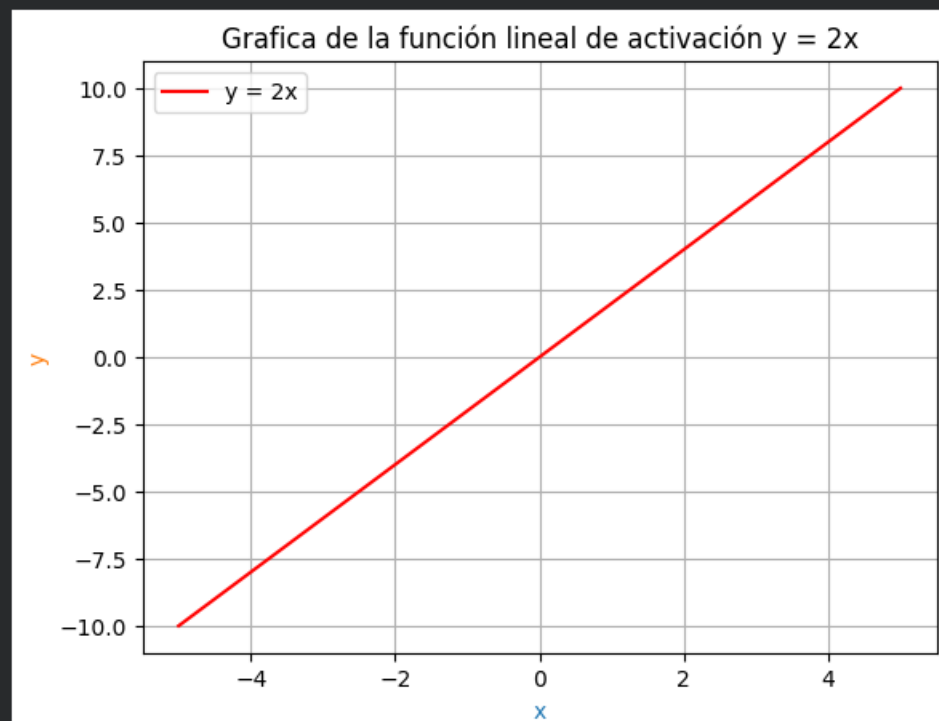
Activación Lineal

La función de activación lineal está definida por $f(x) = ax$. Tenga en cuenta que $f'(x) = a$, para todos $x \in \mathbb{R}$. Tenga en cuenta que $-\infty \leq f(z) \leq \infty$.

La función de activación lineal se utiliza en un solo lugar, es decir, la capa de salida. El efecto de esta función de activación es simplemente escalar los valores provenientes de la última capa oculta, para que se ajusten a los valores objetivo.

```
x = np.linspace(-5,5,100)
y = 2 * x

plt.plot(x, y, '-r', label = 'y = 2x')
plt.title('Grafica de la función lineal de activación y = 2x')
plt.xlabel('x', color = '#1f77b4')
plt.ylabel('y', color = '#ff7f0e')
plt.legend(loc = 'upper left')
plt.grid()
plt.show()
```



Funciones de activación tipo sigmoide

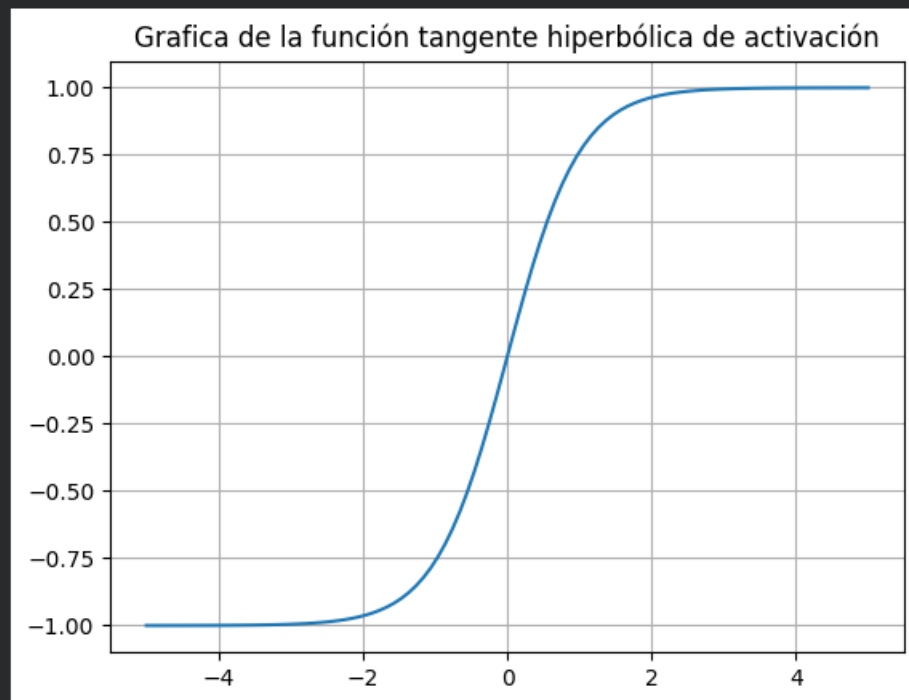
Son funciones cuyo gráfico tiene una forma de "S" alargada. Hay varios tipos de funciones sigmoideas. Las más utilizadas son las funciones logísticas y tangentes hiperbólicas.

Esta función es definida por

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

para $x \in \mathbb{R}$.

```
def tanh(x):  
    return (2/(1+np.exp(-2*x))-1)  
  
x = np.linspace(-5, 5, 100)  
y = tanh(x)  
  
plt.title('Grafica de la función tangente hiperbólica de activación')  
plt.plot(x, y)  
plt.grid()  
plt.show()
```



Por lo general, $f(x)$ se usa en la capa de salida de una clasificación binaria, donde el resultado es 0 o 1, ya que el valor de la función sigmoidea se encuentra entre 0 y 1, por lo que el resultado se puede predecir fácilmente como 1 si el valor es mayor que 0,5 y 0 en caso contrario.

Función de activación Tangente hiperbólica

La activación que funciona casi siempre mejor que la función sigmoidea es la función tanh también conocida como función tangente hiperbólica. En realidad, es una versión matemáticamente modificada de la función sigmoidea. Ambos son similares y pueden obtenerse una de la otra entre sí.

La función Tanh está definida por:

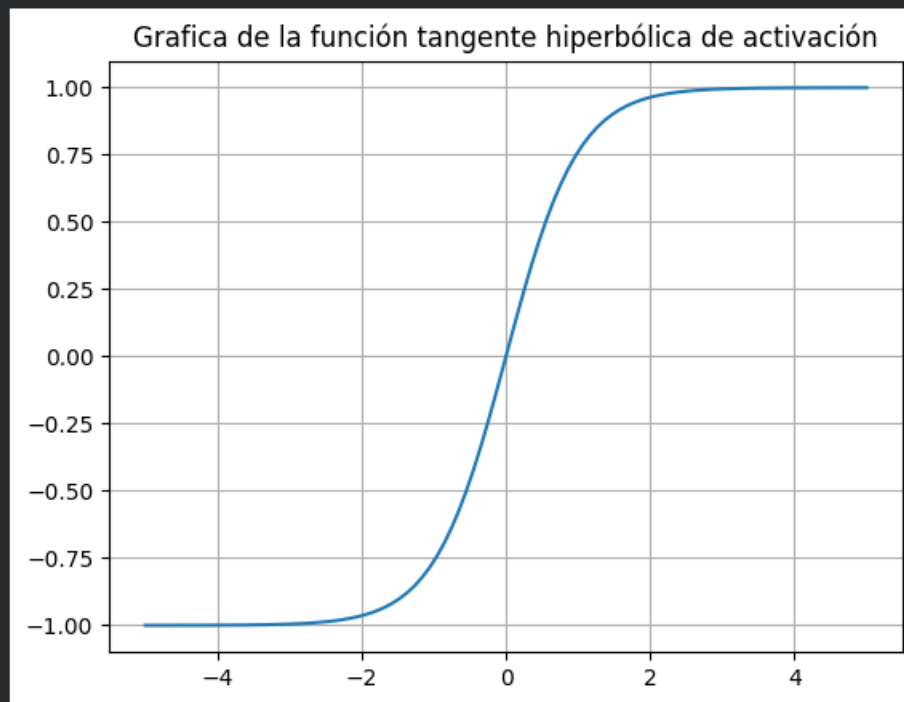
$$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$

para $x \in \mathbb{R}$.

```
def tanh(x):  
    return (2/(1+np.exp(-2*x))-1)
```

```
x = np.linspace(-5, 5, 100)  
y = tanh(x)
```

```
plt.title('Grafica de la función tangente hiperbólica de activación')  
plt.plot(x, y)  
plt.grid()  
plt.show()
```



Tanh se usa generalmente en capas ocultas de una red neuronal, ya que sus valores se encuentran entre -1 a 1, por lo que la media de la capa oculta resulta ser 0 o muy cerca de ella, lo que ayuda a centrar los datos al acercar la media a 0. Esto facilita mucho el aprendizaje de la siguiente capa.

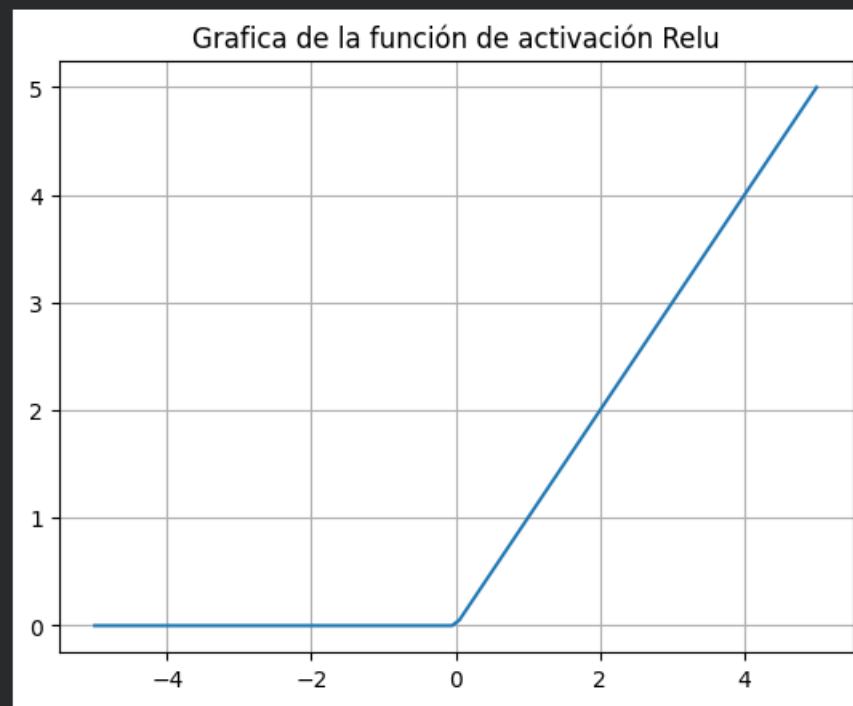
Función de activación Unidad de rectificación lineal (ReLU)

ReLU es la función de activación más utilizada actualmente. Implementada principalmente en capas ocultas de la red neuronal. ReLu se define como

$$f(x) = \text{ReLU}(x) = \max(0, x)$$

para $x \in \mathbb{R}$.

```
def Relu(x):  
    y = np.zeros(x.shape)  
    return np.ndarray.flatten(np.max([[y],[x]], axis = 0))  
  
x = np.linspace(-5, 5, 100)  
y = Relu(x)  
  
plt.title('Grafica de la función de activación Relu')  
plt.plot(x, y)  
plt.grid()  
plt.show()
```



ReLU es menos costosa computacionalmente que Tanh y sigmoide porque involucra operaciones matemáticas más simples. A la vez, solamente se activan unas pocas neuronas, lo que hace que la red sea dispersa, lo que la hace eficiente y fácil de calcular. En palabras simples, Relu permite a la red aprender mucho más rápido que la funciones sigmoide y Tanh.

Funcion de activación Softmax

La función softmax también es un tipo de función sigmoidea, pero es útil cuando intentamos manejar problemas de clasificación.

Generalmente se usa cuando se intenta manejar múltiples clases. La función softmax comprime las salidas para cada clase a valores entre 0 y 1 al dividir cada valor por la suma de las salidas (modificadas mediante una función exponencial).

La función softmax se usa idealmente en la capa de salida del clasificador donde realmente estamos tratando de alcanzar las probabilidades para definir la categoría que corresponde de cada entrada. Formalmente la función softmax es definida como sigue-

Sea $x = (x_1, \dots, x_n)^t \in R$. Softmax es una función $S: R^n \rightarrow R^n$ definida por

$$S(x): \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \rightarrow \begin{pmatrix} S_1 \\ \vdots \\ S_n \end{pmatrix}$$

$$\text{en donde } S_j = \frac{e^{x_j}}{\sum_{k=1}^N e^{x_k}}$$

Dado que S es una función vectorial de R^n a R^n , la derivada es la matriz jacobiana dada por

$$DS = \begin{pmatrix} D_1 S_1 & \cdots & D_1 S_n \\ \vdots & \ddots & \vdots \\ D_n S_1 & \cdots & D_n S_n \end{pmatrix}$$

Luego de unos simples cálculos se llega a que $D_j S_i = S_i(\delta_{ij} - S_j)$

en donde δ_{ij} es llamado el delta de Kroneker definido por

$$\delta_{ij} = \begin{cases} 1, & \text{si } i = j \\ 0, & \text{si } i \neq j \end{cases}$$

```
def Soft(x):  
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))  
    return e_x / e_x.sum(axis=-1, keepdims=True)  
  
x = np.linspace(-5, 5, 100)  
y = tanh(x)  
  
plt.title('Grafica de la función Softmax de activación')  
plt.plot(x, y)  
plt.grid()  
plt.show()
```

